

Análise de Código Java

BackpressureStream — Controle de Fluxo com Filas Bloqueantes

1. Objetivo Geral

A classe **BackpressureStream** implementa um padrão *producer-consumer* com **backpressure**: o produtor gera números inteiros e coloca-os numa fila de capacidade limitada, enquanto o consumidor os retira e processa em lotes. O objetivo é evitar que o produtor sobrecarregue o sistema quando o consumidor é mais lento.

2. Lógica e Algoritmo

O produtor usa **buffer.put(item)**, que *bloqueia* automaticamente quando a fila está cheia — é aqui que o backpressure é exercido. O consumidor usa **buffer.poll(200, MILLISECONDS)**, que aguarda até 200 ms por um elemento; se a fila estiver vazia, termina o lote. No método **main**, o produtor é lançado numa thread separada e, após 100 ms, o consumidor processa até 5 itens (**batchSize**), demonstrando o fluxo controlado.

3. Conceitos e Funcionalidades Java

Concorrência: uso de Thread para paralelismo entre produtor e consumidor. **BlockingQueue / LinkedBlockingQueue**: estrutura thread-safe com capacidade limitada, núcleo do mecanismo de backpressure. **Interface funcional Consumer<T>**: permite passar o handler de consumo como lambda. **TimeUnit**: timeout parametrizado na operação de poll.

4. Complexidade

Cada operação de **put** e **poll** na **LinkedBlockingQueue** é **O(1)**. O ciclo de produção é **O(n)** onde n é o número de elementos; o consumo por lote é **O(batchSize)**. O espaço ocupado pela fila é **O(bufferSize)**, fixo e controlado.

5. Decisões de Design e Limitações

A capacidade da fila (**bufferSize=3**) garante que o produtor pare quando o consumidor não acompanha, evitando *out-of-memory*. Contudo, o consumidor apenas processa **um lote** no **main**, pelo que elementos produzidos após esse lote ficam por consumir. Numa solução de produção, o consumo deveria ocorrer em loop ou em thread dedicada, e seria necessário um mecanismo explícito de paragem (ex.: poison pill ou flag de terminação).